

REKAYASA PERANGKAT LUNAK

Modul Pembelajaran: Analisis Kebutuhan Perangkat Lunak

Penjelasan komprehensif mengenai rekayasa kebutuhan (requirements engineering), metode penggalan data, serta klasifikasi kebutuhan fungsional dan non-fungsional.

Contents

1	Pemahaman Rekayasa Kebutuhan	2
1.1	Apa itu Rekayasa Kebutuhan?	2
1.2	Kenapa Kita Butuh Rekayasa Kebutuhan?	2
2	Metode Penggalan Kebutuhan	3
3	Klasifikasi Tingkatan Kebutuhan	4
3.1	1. User Requirement (Kebutuhan Pengguna)	4
3.2	2. System Requirement (Kebutuhan Sistem)	4
4	Kebutuhan Fungsional vs Non-Fungsional	5
4.1	Kebutuhan Fungsional (Functional Requirement)	5
4.2	Kebutuhan Non-Fungsional (Non-Functional Requirement)	5
5	Klasifikasi Mendalam Kebutuhan Non-Fungsional	6

1 Pemahaman Rekayasa Kebutuhan

1.1 Apa itu Rekayasa Kebutuhan?

Dalam proses membangun sebuah perangkat lunak, tahap yang paling rawan dan menentukan kesuksesan proyek adalah tahap awal: **Rekayasa Kebutuhan** (*Requirements Engineering*).

Secara definisi, rekayasa kebutuhan adalah proses pembentukan layanan-layanan (fungsionalitas) yang dibutuhkan oleh *customer* (pelanggan) dari sebuah perangkat lunak, beserta batasan-batasan (kendala teknis maupun operasional) di mana sistem tersebut nantinya beroperasi dan dikembangkan.

Singkatnya, ini adalah proses "menerjemahkan bahasa keinginan pelanggan" menjadi "bahasa teknis" yang siap dieksekusi oleh tim pemrogram.

1.2 Kenapa Kita Butuh Rekayasa Kebutuhan?

Alasannya sangat sederhana dan mendasar: **karena kita membangun software untuk Customer (Pelanggan).**

Kutipan Head First:

"If your customer is not happy with your software, that means you build a wrong software."
(Jika pelangganmu tidak senang dengan perangkat lunakmu, itu berarti kamu telah membangun perangkat lunak yang salah.)

Penyebab paling umum kegagalan proyek *software* adalah **Poor Communication** (Komunikasi yang buruk) antara tim pengembang dan pelanggan. Seringkali pelanggan meminta A, namun *developer* menangkapnya sebagai B, dan akhirnya membangun sistem C. Oleh karena itu, seorang *Software Engineer* yang baik dituntut **harus bisa menjadi pendengar yang baik** sekaligus **pemberi saran yang baik**.

Namun perlu diingat, *developer* juga harus hati-hati dengan apa yang disebut "*Fool Customer*"—yaitu pelanggan yang sering kali tidak tahu persis apa yang mereka butuhkan atau permintaannya tidak masuk akal secara teknis. Di sinilah peran "pemberi saran yang baik" diuji.

2 Metode Penggalan Kebutuhan

Lalu, bagaimana cara kita "merekayasa" atau menggali kebutuhan pelanggan tersebut? Walaupun masing-masing model proses SDLC (seperti Waterfall, Agile, atau Prototype) punya pendekatan tersendiri dalam rekayasa kebutuhan, namun tujuannya tetap sama: **mendapatkan daftar (list) kebutuhan yang jelas pada perangkat lunak yang akan dibangun.**

Ada empat metode utama yang biasa digunakan untuk menggali kebutuhan (*requirements elicitation*):

1. **Wawancara (Interview):** Ini adalah komunikasi verbal langsung (tatap muka atau virtual) untuk mendapatkan informasi secara mendalam dari satu atau sekelompok orang (klien/pengguna akhir).
2. **Kuesioner (Questionnaire):** Komunikasi dalam bentuk pertanyaan tertulis yang dibagikan dan diberikan kepada *customer*. Sangat cocok digunakan jika kita butuh data dari ratusan atau ribuan calon pengguna sekaligus secara masif.
3. **Observasi (Observation):** Peninjauan secara langsung dan penilaian tentang apa yang sebenarnya terjadi pada suatu proses bisnis di lapangan. Kadang pelanggan tidak bisa menjelaskan cara kerjanya, jadi *developer* harus melihat langsung ke lokasi.
4. **Analisa Dokumen Manual:** Pencarian, pemahaman, dan pengkajian tentang dokumen-dokumen manual (kertas, form, laporan cetak) yang saat ini ada pada suatu kasus dan nantinya akan didigitalisasi ke dalam *software*.

3 Klasifikasi Tingkatan Kebutuhan

Kebutuhan pada *software* tidak bisa langsung ditulis dengan bahasa kode. Ia dimulai dari domain masalah bahasa manusia, yang kemudian dipecah menjadi bahasa teknis.

3.1 1. User Requirement (Kebutuhan Pengguna)

Dimulai dari domain masalah yang dikenal sebagai *user requirement*. Ini adalah tentang **apa yang user harapkan untuk bisa dilakukan pada suatu perangkat lunak**.

- Biasanya dituliskan dalam bentuk *User Requirement Document* atau bisa juga disampaikan secara kasual dalam bentuk *user stories*.
- **Karakteristik:** Kedetailan informasinya tidak terlalu detail, target utamanya adalah pengguna sistem yang tidak mempunyai pengetahuan teknis yang dalam, dan bentuk informasinya berupa bahasa natural.

3.2 2. System Requirement (Kebutuhan Sistem)

Dari *User Requirement*, daftar keinginan pelanggan tadi akan didetailkan ke dalam bentuk solusi domain yang dikenal sebagai *System Requirement*.

- **Karakteristik:** Kedetailan informasinya sangat detail, target penggunaannya adalah para *Developer* (pengembang), dan bentuk informasinya bisa berupa grafik, bahasa teknis, format kode, tabel kebutuhan, atau diagram teknis.

Contoh Pemetaan:

- **User Requirement:** "Petugas perpustakaan harus bisa menambahkan data buku ke dalam sistem sesuai aturan".
- **System Requirement Specification:**
 - Data buku yang disimpan antara lain: kode ISBN, judul buku, penerbit, pengarang.
 - ISBN harus diisi dengan format: xxx-999-xxx-999.
 - Tidak boleh ada data yang dikosongkan pada form data buku.
 - Untuk mengisi data pengarang, petugas perpustakaan memilih dari data pengarang yang sudah ada di *database*.
 - Sistem bisa melakukan pengisian data buku secara otomatis dengan *crawling*/mengambil data API dari `amazon.com`.

4 Kebutuhan Fungsional vs Non-Fungsional

Apapun level dokumennya (User maupun System Requirement), **sebuah kebutuhan yang baik harus terukur dan dapat diuji**. Kebutuhan ini secara fundamental dibagi menjadi dua tipe besar: Fungsional dan Non-Fungsional.

4.1 Kebutuhan Fungsional (Functional Requirement)

Kebutuhan fungsional adalah **pernyataan dari layanan (fitur) sistem yang harus disediakan**. Ini menjelaskan bagaimana sistem harus bereaksi terhadap *input* tertentu, dan bagaimana sistem harus berperilaku dalam situasi tertentu. Intinya: apa yang *software* **harus bisa lakukan**.

Contoh Kebutuhan Fungsional:

- Pengguna bisa menambahkan komentar pada status yang muncul di *timeline*.
- Sistem menyediakan filter foto "Clarendon" yang bisa dipilih pengguna.
- Pengguna bisa *me-mention* (menyebutkan) teman pada *tweet* yang dibuat.
- Mahasiswa bisa memilih kelas KRS sesuai dengan pilihan kelas yang disediakan.

4.2 Kebutuhan Non-Fungsional (Non-Functional Requirement)

Kebutuhan non-fungsional adalah **batasan-batasan** dari layanan-layanan (fungsional) sebuah sistem. Walaupun bernama "non-fungsional", kebutuhan ini **tetap wajib dipenuhi** agar aplikasi bisa beroperasi dengan baik di dunia nyata. Ini meliputi batasan waktu respon, batasan kapasitas, serta batasan-batasan dari *environment* pengguna. Intinya: bagaimana *software* harus **berperforma** dan **merespon keadaan**.

Kebutuhan non-fungsional seringkali dinyatakan oleh pelanggan secara tidak terukur (mengambang). Tugas *Software Engineer* adalah merubahnya menjadi ukuran yang pasti.

Contoh (Tidak Terukur)	Pernyataan Pelanggan	Spesifikasi Kebutuhan Terukur (Developer)
"Sistem yang dibangun harus <i>user-friendly</i> "	user-	"Antarmuka yang dibangun harus berbentuk <i>menu-driven</i> , dan menggunakan komponen UI yang tepat seperti <i>radio button</i> , <i>check box</i> , dll."
"Semua tampilan harus muncul dengan cepat di layar"		"Ketika user mengakses tampilan, halaman akan <i>ter-render</i> muncul di layar dalam waktu kurang dari 3 detik"
"Usaha untuk login harus dibatasi agar aman"		"Seorang user hanya mendapatkan kesempatan maksimal 3 kali salah memasukkan <i>password</i> login pada sistem"

Table 1: Konversi Pernyataan Non-Fungsional Menjadi Terukur

5 Klasifikasi Mendalam Kebutuhan Non-Fungsional

Kebutuhan non-fungsional memiliki hirarki cabang yang sangat luas. Secara umum terbagi menjadi tiga kategori besar: **Product Requirement**, **Organizational Requirement**, dan **External Requirement**.

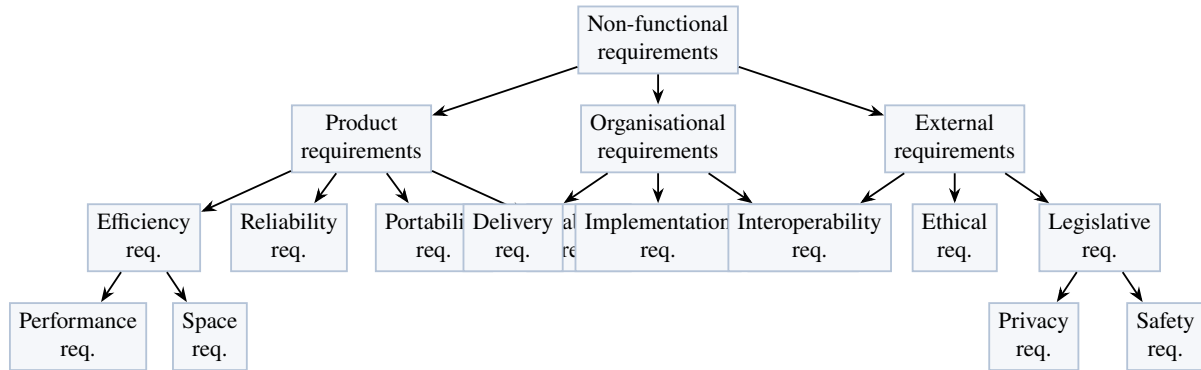


Figure 1: Hirarki Klasifikasi Kebutuhan Non-Fungsional.

Berikut adalah beberapa parameter properti yang biasa dijadikan acuan agar kebutuhan non-fungsional produk (Product Requirement) menjadi valid dan terukur:

- **Kecepatan (Performance):** Diukur dengan besaran transaksi yang diproses per detik, waktu respon terhadap perintah pengguna (waktu layanan < 2 sekon), atau waktu *refresh* layar.
- **Ukuran (Space):** Diukur dalam satuan K Bytes, atau jumlah konsumsi minimum RAM. Misal: Penyimpanan harus hemat, ukuran file unggahan < 10 MB.
- **Kemudahan Penggunaan (Usability):** Diukur dari waktu pelatihan staf untuk menguasai sistem, atau jumlah menu *Help* (bantuan) yang disediakan.
- **Reliabilitas (Keandalan):** Diukur berdasarkan rata-rata waktu terjadinya kegagalan (*Mean Time to Failure*), probabilitas sistem tidak bisa diakses, jumlah kegagalan, dan *Availability* (ketersediaan server 99.9%).
- **Robustness (Ketahanan):** Diukur dari waktu yang dibutuhkan untuk *restart/recovery* ketika terjadi *crash*, persentase kegagalan *event*, dan probabilitas hilangnya data (*data loss*) akibat *crash*.
- **Portability:** Diukur dari persentase kode (statement) yang berhasil dieksekusi lintas *platform* OS. Misal: "Sistem harus bisa berjalan di Windows dan Linux tanpa modifikasi".